Full Project Report — Architecture, ML Pipeline & API

Table of Contents

1 Project Overview

TruthLens is a production-grade misinformation intelligence platform built as a portfolio project for Devkriti'26. Unlike trivial classifiers, TruthLens is designed as a full software product: a modular ML pipeline, REST API, explainability layer, RAG fact-check engine, and a browser-based UI.

The project demonstrates end-to-end ML engineering - from raw dataset ingestion to a live, explainable prediction served over HTTP, complete with fact-check evidence retrieved from a local knowledge base.

> *Key differentiator: TruthLens does not just output fake/real. It tells the user WHY - via highlighted flagged phrases and retrieved supporting evidence.*

Core Capabilities

- Multi-tier inference: classical SVM, BiLSTM/GRU deep learning, DistilBERT transformer
- Automatic routing: short text (<200 chars) goes to headline-tuned SVM; longer text to full-text GRU
- Uncertain-band logic (35-65% probability zone returns 'uncertain' instead of forcing a label)
- Gradient saliency XAI for neural models; coefficient extraction for SVM
- Local RAG engine retrieves top-3 relevant fact-check articles via TF-IDF cosine similarity
- FastAPI REST backend with automatic OpenAPI documentation
- React+Babel frontend - no build step, works directly in the browser
- Dockerized - one command deploys the full stack
- 43 passing pytest tests covering every module

2 Problem Statement

Misinformation spreads faster than corrections. Existing fact-checking tools are either manual (too slow) or black-box binary classifiers (untrustworthy). TruthLens addresses three concrete problems:

- Speed: Automated ML inference in milliseconds vs. hours for manual fact-checking.
- Explainability: Users need to understand what triggered the classification - not just a score.
- Evidence: Classification alone is insufficient; retrieved supporting evidence builds user trust.

Dataset

Name	WELFake_Dataset.csv
Size	72,134 labeled articles
Classes	0 = Real, 1 = Fake
Columns	title, text, label
Source	Verma et al., 2021 (IEEE TITS) - merged from 4 popular datasets

3 System Architecture

TruthLens follows a layered pipeline architecture where each component has a single responsibility and can be replaced independently.

```
Raw Text Input (HTTP POST /predict) | v Preprocessing (lowercasing, URL removal, dateline stripping) | |---> [classical] TF-IDF Vectorizer --> Calibrated SVM | +--> Coefficient XAI | |---> [deep_learning] Vocabulary Tokenizer --> Bidirectional GRU | +--> Gradient Saliency XAI | +---> [transformer] HuggingFace Tokenizer --> DistilBERT Fine-tuned +--> Gradient Saliency XAI | v Uncertain-Band Thresholding (35-65% = 'uncertain') | v Local RAG Engine (TF-IDF cosine similarity) Retrieves top-3 fact-check articles | v PredictionResponse (JSON) | v TruthLens UI (React + Babel)
```

Automatic Model Routing

When model_type='classical' is requested, the API inspects the character length of the input text. Inputs shorter than 200 characters are treated as headlines and routed to a headline-specific SVM. Longer inputs go to the full-article SVM. This dual-model approach avoids performance degradation on very short, ambiguous headlines.

4 ML Pipeline - Stage by Stage

Stage 1 - Baseline (TF-IDF + Logistic Regression)

The baseline establishes a performance floor. Text is cleaned, tokenized, and transformed into a TF-IDF matrix. A Logistic Regression with L2 penalty is trained and evaluated. This stage takes minutes to run and achieves ~94% accuracy - a surprisingly strong baseline that highlights the power of bag-of-words representations.

- TF-IDF: max 100,000 features, (1,2)-gram range, sublinear TF scaling
- Logistic Regression: C=1.0, max_iter=1000, solver='lbfgs'
- Accuracy: ~94.1%

Stage 2 - Classical Model Comparison

Five classical models are compared under identical preprocessing: Logistic Regression, Naive Bayes, Linear SVM, Random Forest, and XGBoost. A Calibrated Linear SVM is selected for deployment based on F1 score, calibration quality, and inference speed.

Model	Accuracy	F1 Score	ROC AUC
Logistic Regression	94.1%	94.0%	0.990
Naive Bayes	89.3%	89.1%	0.963
Linear SVM (deployed)	96.2%	96.1%	0.994
Random Forest	93.8%	93.7%	0.988
XGBoost	94.7%	94.6%	0.991

Stage 3 - Deep Learning (Bidirectional GRU)

A custom vocabulary is built from the training corpus and used to tokenize text into integer sequences padded to 300 tokens. A Bidirectional GRU processes the sequence in both directions. The final hidden state is concatenated, passed through dropout, and projected to binary output via sigmoid activation.

- Embedding dim: 128 | Hidden dim: 256 (128 each direction) | Dropout: 0.3
- Optimizer: Adam (lr=3e-4) | Batch size: 64 | Epochs: 10 with early stopping
- Accuracy: ~97.1% | F1: ~97.0%

Stage 4 - Transformer (DistilBERT)

DistilBERT-base-uncased is fine-tuned for binary classification using HuggingFace Transformers. The [CLS] token representation is fed to a linear classification head. Mixed precision (fp16) training is used when a GPU is available.

- Model: distilbert-base-uncased (66M parameters)

- Max sequence length: 256 tokens | Learning rate: 2e-5 | Warmup steps: 500
- Accuracy: ~98.2% | F1: ~98.1%

5 Explainable AI (XAI)

TruthLens provides human-readable explanations for every prediction. The strategy varies by model family.

Classical Models - Coefficient Extraction

For TF-IDF SVM models, the trained weight vector is a direct measure of feature importance. The top-K TF-IDF n-grams with the highest absolute weight contribution towards the predicted class are extracted and returned as flagged phrases. This is deterministic and fast (O(k log k)).

Neural Models - Gradient Saliency

For GRU and DistilBERT models, gradient-based saliency is computed. The gradient of the predicted class score with respect to the input token embeddings is calculated via backpropagation. The L2 norm of the gradient at each token position gives its saliency score. Tokens with the highest saliency are returned as flagged phrases.

> *Both strategies return the same data format - a list of string phrases - so the frontend and API schema are completely agnostic to the model type being explained.*

6 RAG Fact-Check Engine

After classification, TruthLens runs a Retrieval-Augmented Generation (RAG) lookup against a curated local knowledge base of verified fact-check articles. This grounds the prediction in real-world evidence without requiring internet access or an LLM.

How It Works

- Knowledge base: data/fact_checks.json - a JSON array of fact-check documents with title, verdict, source, url, and body.
- At startup: LocalRAG fits a TF-IDF vectorizer on all document bodies and stores the resulting document matrix.
- At inference: the input text is transformed by the same TF-IDF vectorizer, then cosine similarity is computed against all document vectors.
- Top-K (default 3) most similar documents above a threshold are returned as evidence.
- The evidence list is included in the API response JSON.

Why TF-IDF Rather Than a Vector Database?

A dense vector database (FAISS, Pinecone) would require an embedding model and significant memory. For this project's knowledge base size (~200 documents), sparse TF-IDF retrieval achieves equivalent recall with zero external dependencies and sub-millisecond query time.

7 REST API

The API is built with FastAPI and follows the OpenAPI 3.0 specification. All request/response schemas are defined as Pydantic v2 models for runtime validation and automatic documentation generation.

Endpoints

GET /	Health check - returns loaded model names and routing config
POST /predict	Main inference endpoint - classifies text and returns full response
GET /docs	Interactive Swagger UI (auto-generated by FastAPI)
GET /ui	Serves the TruthLens React frontend

POST /predict - Request

```
{ "text": "string (required) - article body or headline", "title": "string (optional) - article title", "model_type": "classical | deep_learning | transformer", "combine_title_text": "bool (default: true)" }
```

POST /predict - Response

```
{ "label": 0 | 1 | -1, "label_name": "real" | "fake" | "uncertain", "fake_probability": 0.97, "real_probability": 0.03, "model_tier": "headline" | "article" | "deep_learning" | "transformer", "model_type": "classical" | "deep_learning" | "transformer", "model_metadata": { "model_name", "trained_at_utc", "dataset", "metrics" }, "flagged_phrases": ["clickbait term", "conspiracy"], "evidence": [ { "title": "...", "verdict": "False", "source": "Snopes", "url": "https://...", "similarity_score": 0.89 } ] }
```

8 TruthLens UI

The frontend is a single-page React application built with Babel standalone - no Node.js, no build step. It is served directly by FastAPI's StaticFiles mount at /ui.

Key UI Features

- Animated scanning progress bar while waiting for the API response
- Verdict card - color-coded (red/green/amber) with large probability display
- Highlighted text panel - flagged phrases are underlined in the input text
- RAG Evidence panel - retrieved fact-check articles with verdict badges
- Model selector - switch between classical, GRU, and transformer at runtime
- Responsive layout for desktop and tablet

9 Model Performance & Benchmarks

Classification Metrics (WELFake, held-out 20% test set)

Model	Accuracy	Precision	Recall	F1	ROC AUC
SVM (classical)	96.2%	96.1%	96.3%	96.1%	0.994
GRU (deep)	97.1%	97.0%	97.2%	97.0%	0.997
DistilBERT (trans)	98.2%	98.2%	98.2%	98.1%	0.999

Inference Latency (single request, CPU)

Model	Median Latency	P99 Latency
SVM (classical)	< 5 ms	< 10 ms
GRU (deep)	~120 ms	~200 ms
DistilBERT (trans)	~450 ms	~700 ms

The GRU provides the best accuracy/latency tradeoff for production and is the default model_type in the API.

10 Tech Stack

Python	3.12+ - primary language
FastAPI	REST API framework with async support and auto OpenAPI docs
Pydantic v2	Request/response schema validation
scikit-learn	TF-IDF, SVM, Logistic Regression, evaluation metrics
PyTorch	BiLSTM/GRU model definition and gradient saliency
HuggingFace	DistilBERT tokenizer and fine-tuning (Trainer API)
NLTK	Stopword removal, tokenization
React + Babel	Frontend - no build step, served from FastAPI
Docker	Containerization and deployment
pytest	Test suite - 43 tests across all modules
YAML	Configuration files - all hyperparameters externalized

11 Project Structure

```
fake-news-detection-system/ |-- app/ | |-- main.py # FastAPI app, lifespan, /predict endpoint | +-- schemas.py # Pydantic request/response models |-- src/ | |-- data/ | | |-- clean.py # Text cleaning and dateline stripping | | |-- load.py # Dataset loading utilities | | +-- preprocess.py# Tokenization and preprocessing pipeline | |-- features/ | | +-- tfidf.py # TF-IDF vectorizer training and persistence | |-- models/ | | |-- baseline.py # Stage 1 Logistic Regression | | |-- classical.py # Stage 2 multi-model comparison | | |-- deep_learning.py # GRU/BiLSTM architecture | | |-- transformer.py # DistilBERT fine-tuning | | |-- inference.py # Unified inference for all model types | | +-- evaluate.py # Metrics computation | |-- explain/ | | +-- explain.py # XAI: coefficient extraction + gradient saliency | +-- rag/ | +-- retriever.py # LocalRAG - TF-IDF fact-check retrieval |-- frontend/ | |-- index.html # TruthLens React UI entry point | +-- TruthLens.jsx # React component tree |-- configs/ # YAML configs for each model stage |-- data/ | |-- raw/ # WELFake_Dataset.csv | +-- fact_checks.json # RAG knowledge base |-- docs/ # Experiment reports per stage |-- notebooks/ # Jupyter notebooks (EDA, baselines, DL, transformers) |-- tests/ # pytest suite (43 tests) |-- utils/ # Config loader, logging, notebook helpers |-- Dockerfile |-- docker-compose.yml +-- README.md
```

12 Setup & Running Locally

1. Install

```
git clone https://github.com/meetcodz/fake-news-detection-system.git cd
fake-news-detection-system pip install -e ".[dev,notebook]"
```

2. Train Models

```
python -m src.models.train # Stage 1 & 2 (classical, fast) python -m src.models.train_deep #
Stage 3 - GRU python -m src.models.train_transformer # Stage 4 - DistilBERT
```

3. Start API

```
uvicorn app.main:app --reload # API: http://127.0.0.1:8000 # Docs: http://127.0.0.1:8000/docs #
UI: http://127.0.0.1:8000/ui
```

4. Docker (one command)

```
docker compose up --build
```

5. Run Tests

```
python -m pytest tests/ -v --basetemp=./tmp_pytest
```

13 Interview Q&A; Guide

The following are the most commonly asked questions in ML engineering interviews for projects of this type, with recommended talking points.

Q: Why did you choose WELFake over other datasets?

WELFake merges four independent datasets, which reduces source bias. With 72k samples it's large enough to evaluate neural models. It also has both title and body columns, letting me explore title-only vs. combined inputs.

Q: What is TF-IDF and why use it as a baseline?

TF-IDF weights each word by how often it appears in a document relative to how common it is across all documents. Common words like 'the' get near-zero weight; rare, discriminative words get high weight. It's interpretable, fast, and achieves surprisingly competitive accuracy - making it an ideal baseline.

Q: Why a Bidirectional GRU instead of LSTM?

GRUs have fewer parameters than LSTMs (no separate cell state), making them faster and less prone to overfitting. The bidirectional setup lets the model use both left and right context, which is important for understanding negation in news text.

Q: How does your XAI work for neural models?

I compute gradient saliency: backpropagate the predicted class score to the input embedding layer, then take the L2 norm of the gradient at each token. High gradient magnitude means that token had high influence. I return the top-K tokens as flagged phrases. No external library - just PyTorch's autograd.

Q: What is RAG and how did you implement it without an LLM?

RAG is the idea of grounding model outputs in retrieved documents. Normally RAG uses a dense embedding model. I implemented a lightweight version using sparse TF-IDF vectors: at startup, all fact-check docs are vectorized. At inference, cosine similarity finds the closest matching documents. No LLM, no API calls.

Q: What is the uncertain-band logic?

A classifier that outputs 0.51 should not be treated the same as one that outputs 0.99. Predictions in the 35-65% probability zone return label=-1 and label_name='uncertain' instead of forcing a real/fake decision. This makes the system more honest about ambiguous inputs.

Q: How is the project structured for maintainability?

Each module has one responsibility (SOLID/SRP). Business logic lives exclusively in src/. The FastAPI app/ layer handles only HTTP concerns. All hyperparameters are in YAML configs - nothing is hardcoded. I can swap the model or change thresholds without touching any Python code.

Q: What would you improve next?

Three things: (1) Replace the local RAG knowledge base with a live fact-checking API (e.g. ClaimBuster) for real-time evidence. (2) Add PostgreSQL persistence to log predictions for drift monitoring. (3) Fine-tune DeBERTa-v3 - it consistently outperforms DistilBERT on NLI tasks and should push F1 to ~99%.